

Add more checks for the validity of refnames

#1672

New issue

Merged

Byron merged 1 commit into `gitpython-developers:main` from `trail-of-forks:robust-refname-checks` on Sep 22, 2023

Conversation 25 Commits 1 Checks 0 Files changed 2 +84 -2



facutuesca commented on Sep 21, 2023

Contributor

• edited

This change adds checks based on the rules described in [the docs](#) in order to more robustly check a refname's validity.



Add more checks for the validity of refnames `46d3d05`



facutuesca mentioned this pull request on Sep 21, 2023

Fix CVE-2023-41040 #1644

Merged



facutuesca commented

Contributor

Author

on Sep 21, 2023 • edited

To add a bit more context: I followed the general approach [mentioned](#) by [@Byron](#) and used by [gitoxide](#), which is a `for` loop that iterates over each character, with memory of the previous two characters seen. This is faster than the naive approach (since we minimize the amount of times we iterate over the refname string), at the cost of some readability.

For comparison, here's the naive approach, where the logic separation matches [the docs](#) (one rule per `if` condition):

```
def _check_ref_name_valid_naive(ref_path: Path)
    # Based on https://git-scm.com/docs/git-check-ref
    if any([component.startswith(".") or component.e
        raise ValueError(f"Invalid reference '{ref_p
    elif ".." in str(ref_path):
        raise ValueError(f"Invalid reference '{ref_p
```

Reviewers



Byron



EliahKagan



Assignees

No one assigned

Labels

None yet

Milestone

v3.1.37 - Bugfixes

Development

Successfully merging this pull request may close these issues.

None yet

4 participants



```

elif any([ord(c) < 32 or ord(c) == 127 or c in [
    raise ValueError(
        f"Invalid reference '{ref_path}': refere
    )
elif any([c in ["?", "*", "["] for c in ref_path
    raise ValueError(
        f"Invalid reference '{ref_path}': refere
    )
elif ref_path.startswith("/") or ref_path.endswi
    raise ValueError(f"Invalid reference '{ref_p
elif ref_path.endswith("."):
    raise ValueError(f"Invalid reference '{ref_p
elif "@" in ref_path:
    raise ValueError(f"Invalid reference '{ref_p
elif ref_path == "@":
    raise ValueError(f"Invalid reference '{ref_p
elif "\\" in ref_path:
    raise ValueError(f"Invalid reference '{ref_p

```

The naive approach is IMO more readable, but around half as fast as the one in the PR. Although, for reference, in my MacBook M1 Pro, for a rename 25 characters long:

- Time to validate using approach in PR: 2.48us
- Time to validate using naive approach: 4.69us

So we are talking about minimal amounts either way. I'll leave the choice of which algorithm to use up to the maintainers.



Byron requested changes
on Sep 21, 2023

[View reviewed changes](#)

Byron left a comment

Member

Thanks a million, I love this implementation!

Strangely enough, I find the faster version (the one here) more readable as well and would want to keep it for that reason alone.

There is one issue I see that might be hard to solve, but it's time to at least try. It's the general problem of how to interact with paths without running into decoding problems (i.e. Python tries to decode a path as decoding X, and fails, even though it's a valid filesystem path). Maybe [@EliahKagan](#) also has ideas regarding this topic.



1

git/refs/symbolic.py

166	+	# Based on the rules descri
167	+	previous: Union[str, None] =
168	+	one_before_previous: Union[s

```
169 + for c in str(ref_path):
```



Byron on Sep 21, 2023

Member

Is there a way to avoid converting to `str`? I assume this tries to decode `ref_path` with the current string encoding, which changes depending on the interpreter or user configuration and generally causes a lot of trouble.



EliahKagan on Sep 22, 2023

Member

• edited ▼

Unless this PR worsens that problem in some way, which I believe it does not, I would recommend it be fixed separately and later. The code this is replacing already had:

[GitPython/git/refs/symbolic.py](#)

Lines 171 to 172 in [d40320b](#)

```
171         if ".." in str(ref_path):
172             raise ValueError(f"Invalid
reference '{ref_path}'")
```

But actually even that neither introduced nor exacerbated the problem. From the commit prior to [#1644](#) being merged:

[GitPython/git/refs/symbolic.py](#)

Lines 164 to 174 in [830025b](#)

```
164         @classmethod
165         def _get_ref_info_helper(
166             cls, repo: "Repo",
167             ref_path: Union[PathLike, None]
168             ) -> Union[Tuple[str,
169             None], Tuple[None, str]]:
170             """Return: (str(sha),
171             str(target_ref_path)) if
172             available, the sha the file at
173             rela_path points to,
174             or None. target_ref_path is the
175             reference we
```

Note how `str(ref_path)` was passed to `os.path.join`, which when given `str`s returns a `str`, thus a `str` was being passed to `open`. Note also that, while this `str` call was actually redundant ([os.path.join](#) accepts [path-like objects](#) since Python 3.6), even *it* was not the cause of `str` and not `bytes` being used. The annotation on `ref_path` is `Union[PathLike, None]`, where `PathLike` is:

[GitPython/git/types.py](#)

Line 43 in [830025b](#)

```
43      PathLike = Union[str,  
      "os.PathLike[str]]"
```

Where both alternatives-- `str` and `os.PathLike[str]` --represent text that has already been decoded.

So unless I'm missing something--which I admit I could be--I don't think it makes conceptual sense to do anything about that in this pull request. Furthermore, unless the judgment that [CVE-2023-41040](#) was a security vulnerability was mistaken, or something about the variation explicated in [#1644 \(comment\)](#) is less exploitable, it seems to me that this pull request is fixing a vulnerability. Assuming that is the case, then I think this should avoid attempting to make far-reaching changes beyond those that pertain to the vulnerability, and that although reviewing these changes for correctness *should not be rushed*, other kinds of delays should be mostly avoided. With good regression tests included, as seems to be the case, the code could be improved on later in other ways.



Byron [on Sep 22, 2023](#)

Member

Thanks a lot for the thorough assessment, I wholeheartedly agree.

The 'how to handle paths correctly' issue is definitely one of the big breaking points in GitPython, but maybe, for other reasons, this wasn't ever a problem here.

Knowing this is on your radar, maybe one day there will be a solution to it. `gitoxide` already solves this problem, but it's easier when you have an actual type system and a standard library that makes you aware every step of the way.



EliahKagan [on Sep 22, 2023](#)

Member

Is the ultimate goal to support both `str`-based and `bytes`-based ref names and paths?



Byron [on Sep 22, 2023](#)

Member

The goal is correctness, and it's vital that one doesn't try to decode paths to fit some interpreter-controlled encoding. Paths are paths, and if you are lucky, they can be turned into bytes. On Unix, that's always possible and a no-op, but on windows it may require a conversion. It's just the question how these things are supposed to work in python.



EliahKagan on Sep 22, 2023

Member

Does this relate (conceptually, I mean) to the issue in [rust-lang/rust#12056](#)?



Byron on Sep 22, 2023

Member

A great find :) - yes, that's absolutely related. `gitoxide` internally handles git-paths as bundles of bytes without known encoding, and just like `git`, it assumes at least ASCII. Conversions do happen but they all go through [gix-path](#) to have a central place for it.

Doing something like it would be needed here as well, even though I argue that before that happens universally, there should be some clear definition of what GitPython is supposed to be.

When I took it over by contributing massively, just like you do now, I needed more control for the use-case I had in mind, and started implementing all these sloppy pure-python components that don't even get the basics right. With that I turned GitPython into some strange hybrid which I think didn't do it any good besides maybe being a little faster for *some* usecases. After all, manipulating an index in memory has advantages, but there are also other ways to do it while relying on `git` entirely.

Maybe this is thinking a step too far, but I strongly believe that the true benefit of GitPython is to be able to call `git` in a simple manner and to be compliant naturally due to using `git` directly. This *should* be its identity.

But then again, it's worth recognizing that changing the various pure-python implementations to us `git` under the hood probably isn't possible in a non-breaking way.

Another avenue would be to try and get the APIs to use types that don't suffer from encoding/decoding issues related to Paths, and then one day make the jump to replacing the pure-python implementations with the python bindings of `gitoxide`.



 **Byron** added this to the **v3.1.37 - Bugfixes** milestone on Sep 22, 2023



 **Byron** merged commit `e98f57b` into `gitpython-developers:main` on Sep 22, 2023



Byron commented on Sep 22, 2023 • edited ▾

Member

It looks like CI improved and now that the PR was merged, it failed CI due to a lint: <https://github.com/gitpython-developers/GitPython/actions/runs/6271134895/job/17030195508#step:4:122> . A quick fix will be appreciated.

Edit: I quickly fixed it myself - it seems like sometimes I forget that I am still able to edit text, despite it being python.



1



 **facutuesca** deleted the `robust-refname-checks` branch 3 years ago



EliahKagan commented on Sep 22, 2023

Member

• edited ▾

@Byron Because the forthcoming 3.1.37 release that will include this patch will be a security fix, either the existing advisory/CVE should be updated with a correction (both a note and the version change), or a new CVE should be created for the variant of the vulnerability reported at [#1644 \(comment\)](#). I am not sure which of those things should be done here.

Usually I would lean toward regarding such things as new bugs meriting new advisories/CVEs, which is also what I see more often. But I do not know that that's the best approach here, because the variant of the exploit where an absolute (or otherwise non-relative) path is used does seem to match the description in the summary section of [CVE-2023-41040](#) even though it doesn't resemble any of the examples. *To be clear, I don't mean that this situation is necessarily ambiguous, but instead that I do not have the knowledge and experience to know how it ought to be handled.* Either way, this need not delay the release, of course.

(Sorry if you're already on top of the CVE/advisory matter and this comment is just noise.)



Byron commented on Sep 22, 2023

Member

Thanks for the hint, it's appreciated!

I think it's fair to say that I am not on top of CVEs and that I have no intention to be - even though this sounds harsh it's just the current reality. But thus far members of the community picked up the necessary work around CVEs which I definitely appreciate if this would keep happening.



Byron commented on Sep 22, 2023

Member

A new release was created:

<https://pypi.org/project/GitPython/3.1.37/>



1



EliahKagan commented on Sep 22, 2023

Member

• edited ▼

Given the [3.1.37](#) release title ("3.1.37 - a proper fix [CVE-2023-41040](#)") I'm thinking this is intuitively being regarded as fixing the originally reported vulnerability, so perhaps that advisory should be updated, rather than a new one created? I am still not sure.

As noted in [#1638 \(comment\)](#), you ([@Byron](#)) could update the local advisory. If you do, a PR could then also be opened on <https://github.com/github/advisory-database> (where [github/advisory-database#2690](#) was opened) to change the global advisory accordingly. I don't know if there's anything else that would need to be done.

@stsewd Do you have any opinion about what ought to be done here? Would you have any objection to the local advisory being edited this way? Would you instead prefer that this variant, where an absolute path is used, be regarded as a related but separate vulnerability altogether? (I know **@Byron** can edit the advisory, but I wanted to check in case you had an opinion on this.)

One source of my hesitancy here is that I think a new CVE may still be needed in this kind of situation. [That seems common](#) (courtesy of [this SO answer](#)).



Byron commented on Sep 22, 2023

Member

As noted in [#1638 \(comment\)](#), you (**@Byron**) could update the local advisory. If you do, a PR could then also be opened on <https://github.com/github/advisory-database> (where [github/advisory-database#2690](#) was opened) to change the global advisory accordingly. I don't know if there's anything else that would need to be done.

A good point - I am still getting used to advisories and the local ones are indeed editable. So that one has been adjusted. I kindly ask somebody else to create a PR for the global database though - it seems GitHub makes it hard/impossible to the use web interface for that.

One source of my hesitancy here is that I think a new CVE may still be needed in this kind of situation.

To me, CVEs are good to create a far-reaching 'ping' to users of GitPython. Some might see it earlier than the new release. To me it's the question on how much time one wants to spend to create such a ping, and judging from the CVE's I have seen, it's quite expensive.



EliahKagan commented on Sep 22, 2023

Member

• edited ▼

A good point

Thanks, but I'm not actually sure if it was a good point. Maybe a new advisory ought to have been, or ought to be, created. I really don't know the proper thing to do here.



Byron commented on Sep 22, 2023

Member

If there is an uproar because of how this was handled, it will be possible to undo changes to the local CVE and create a new one. So I think nothing is lost, and I think it's OK to choose less expensive options in dealing with this.



stsewd commented on Sep 22, 2023

Contributor

Hi, a new CVE/advisory is usually created for this type of situation, and in the description you can put something like "this was a due to an incomplete fix of [link to the other CVE]". I don't oppose to edit the current one, but I guess editing doesn't have the same "ping to everyone to upgrade" effect as a new one.



renovate bot referenced this pull request in allenporter/flux-local on Sep 23, 2023



Update dependency GitPython
to v3.1.37 (#341) ...

Verified

70a5be2



EliahKagan commented on Sep 24, 2023

Member

• edited ▼

Hi, a new CVE/advisory is usually created for this type of situation, and in the description you can put something like "this was a due to an incomplete fix of [link to the other CVE]". I don't oppose to edit the current one, but I guess editing doesn't have the same "ping to everyone to upgrade" effect as a new one.

Thanks!

@Byron Based on this, and also what I am now seeing is the recent history of this practice being followed for GitPython in [CVE-2022-24439/CVE-2023-40267](#), I recommend making a new advisory. Maybe there is some way I can help with this?

However, if for any reason you would still prefer this route not be taken, then I can definitely go ahead and open a PR to update the global advisory with the version change. (I am unsure if that would cause Dependabot to notify users of the security update or not, but I imagine that, if it would not, then a reviewer on the PR would mention that.)

But thus far members of the community picked up the necessary work around CVEs which I definitely appreciate if this would keep happening.

I have three ideas of what I could do, but I don't know what, if any of them, would help or be wanted. This depends, in part, on [what takes up the time for you](#).

1. If the issue is drafting the text of the advisory, I can write a draft and propose that, here, to you. (I considered doing that for this comment, but I figured it would be better to ask first.) You would still have to create the advisory and request the CVE in the same way as you did for [CVE-2023-41040](#).
2. If the issue is the process *after* that, then I might be able to actually request the CVE. Although GitHub is a [CNA](#), I don't think they provide a way to request a CVE except by a maintainer and through the interface you have used. [MITRE](#) is a CNA and I've heard of non-maintainers requesting CVEs from them successfully. However, I am unsure if they would accept such a request from me, because I have no specific connection to this vulnerability (I did not discover, report, analyze, fix, or integrate a fix for it). In addition, if I make the request, then I would first want to ask you some questions about how a situation would arise where someone could exploit this vulnerability without otherwise already being able to open files outside the local repository's `.git` directory, to ensure that I would be able to stand fully by any statements I would make in the request and afterwards. Given that, I am unsure to what extent this option would save you effort.
3. Combination of 1 and 2: I could draft a new advisory, and you could create and publish the new advisory based on my draft (with any modifications you deem appropriate) via the GitHub interface, but *not* request a CVE for it. Even at this point something would have been achieved, I believe, because *within* the GitHub ecosystem (e.g., for Dependabot), I think alerts would be generated once it makes its way into the GitHub Advisory Database. Then I could attempt to request a CVE from some CNA, which if/when assigned could be associated with the advisory.



Byron commented on Sep 24, 2023

Member

@Byron Based on this, and also what I am now seeing is the recent history of this practice being followed for GitPython in [CVE-2022-24439](#)/[CVE-2023-40267](#), I recommend making a new advisory. Maybe there is some way I can help with this?

However, if for any reason you would still prefer this route not be taken, then I can definitely go ahead and open a PR to update the global advisory with the version change. (I am unsure if that would cause Dependabot to notify users of the security update or not, but I imagine that, if it would not, then a reviewer on the PR would mention that.)

Let's try something: updating version numbers is much cheaper than creating a new 'follow-up' CVE, for all sides actually. One could ask in the PR of the version change if notifications will be sent, and if unknown, [@stsewd](#) could probably help to tell as well.

If no notification is sent, you could create a new CVE - you would be able to [do this here in GitPython](#) and from there it can be elevated, along with requesting a global CVE for it - this is easily done through the maintainer interface. The rest we can take from there should it come to that.



EliahKagan commented on Sep 24, 2023

Member

Let's try something: updating version numbers is much cheaper than creating a new 'follow-up' CVE, for all sides actually.

Sounds good; I will do this.

I noticed in the local advisory that, while `3.1.37` is given for patched versions, `<=3.1.34` is still given for affected versions. I recommend changing that `<=3.1.34` here to `<=3.1.36` for consistency, and I'll specify `<=3.1.36` in my proposed change to the global advisory. But if that is no the case and you want that specified differently, please let me know and I'll do differently (or amend the PR if I have already made it).

One could ask in the PR of the version change if notifications will be sent

I'm making the PR through the structured "Suggest improvements" template, in which every field is pretty specific. I'll either include it somewhere if it fits, or otherwise try and add it into the created PR or add a comment with it.

If no notification is sent, you could create a new CVE - you would be able to [do this here in GitPython](#) and from there it can be elevated, along with requesting a global CVE for it - this is easily done through the maintainer interface. The rest we can take from there should it come to that.

Thanks for telling me about that. That is much nicer than the particular specific I had suggested might be used. Of course, I'll still save that for if the above proves insufficient, as you say.



Byron commented [on Sep 24, 2023](#) • edited ▾

Member

I noticed in the local advisory that, while `3.1.37` is given for patched versions, `<=3.1.34` is still given for affected versions. I recommend changing that `<=3.1.34` here to `<=3.1.36` for consistency, and I'll specify `<=3.1.36` in my proposed change to the global advisory. But if that is no the case and you want that specified differently, please let me know and I'll do differently (or amend the PR if I have already made it).

Thanks for the head's up, that's an oversight that is now corrected.

And thanks again for your help!



EliahKagan mentioned this pull request [on Sep 24, 2023](#)

[GHSA-cwvm-v4w8-q58c] Blind local file inclusion [github/advisory-database#2753](#)

Merged



EliahKagan commented [on Sep 24, 2023](#)

Member

• edited ▾

No problem! I've submitted the proposed edit to the global advisory in PR [github/advisory-database#2753](#).



EliahKagan commented [on Oct 11, 2023](#)

Member

[github/advisory-database#2753](#) has been merged and the global GitHub advisory for [CVE-2023-41040](#) has thus been updated.



EliahKagan commented [on Oct 11, 2023](#)

Member

Further update: The change to the global advisory *has* caused Dependabot security alerts to be raised, as desired. For example, [dmvassallo/EmbeddingScratchwork#248](#) is a PR opened automatically to resolve a new Dependabot security alert in a project where GitPython had already been upgraded to the previously listed version. Note that this does not necessarily apply for tools that are less closely coupled to the GitHub ecosystem, and I don't know, for example, if any new [Renovatebot](#) PRs will be generated.

🔥 1



otc-zuul (bot) pushed a commit to opentelekomcloud-infra/eyes_on_docs that referenced this pull request on Oct 25, 2023



[Bump gitpython from 3.1.35 to 3.1.37 \(#41\)](#) Verified [b967bc8](#)



 **EliahKagan** mentioned this pull request on Dec 22, 2023

Fix TemporaryFileSwap regression where file_path could not be Path
[#1776](#)

 Merged



 **data-sync-user** mentioned this pull request on Feb 9, 2025

Bump gitpython from 3.1.32 to 3.1.37
[mozilla/opmon#168](#)

 Open



 **github-actions** (bot) mentioned this pull request on Aug 8, 2025

Critical vulnerabilities detected in dependencies [firefly-oss/sbom-tool#7](#)

 Open